

MVP争夺战

题目描述

在星球争霸篮球赛对抗赛中，最大的宇宙战队希望每个人都能拿到 MVP，MVP 的条件是单场最高得分获得者。

可以并列所以宇宙战队决定在比赛中尽可能让更多队员上场，并且让所有得分的选手得分都相同，然而比赛过程中的每一分钟的得分都只能由某一个人包揽。

输入描述

输入第一行为一个数字 t，表示为有得分的分钟数

- $1 \leq t \leq 50$

第二行为 t 个数字，代表每一分钟的得分 p

- $1 \leq p \leq 50$

输出描述

输出有得分的队员都是 MVP 时，最少得 MVP 得分。

用例

输入	9 5 2 1 5 2 1 5 2 1
输出	6
说明	样例解释 一共 4 人得分，分别都是 6 分 5 + 1， 5 + 1， 5 + 1， 2 + 2 + 2

题目解析

本题和

[LeetCode - 698 划分为k个相等的子集_伏城之外的博客-CSDN博客](#)

属于同一类问题，解法相同，题解请看上面博客：划分k个相等的子集

JavaScript 算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const n = parseInt(await readline());
7      const arr = (await readline()).split(" ").map(Number);
8
9      console.log(solution(arr, n));
10 })();
11
12 function solution(arr, n) {
13     const sum = arr.sort((a, b) => b - a).reduce((p, c) => p + c);
14
15     let count = n;
16     while (count >= 1) {
17         // 由于canPartition方法中会删除arr元素，因此我们不能直接传递arr过去，需要传递arr备份，否则会影响下一次count判断
18         if (canPartition([...arr], sum, count)) {
19             return sum / count;
20         } else {
21             count--;
22         }
23     }
24 }
25
26 function canPartition(arr, sum, count) {
27     if (sum % count) return false;
```

运行

```
28
29   let subSum = sum / count;
30
31   if (subSum < arr[0]) return false;
32
33   while (arr[0] === subSum) {
34     arr.shift();
35     count--;
36   }
37
38   const buckets = new Array(count).fill(0);
39
40   return partition(0, arr, subSum, buckets);
41 }
42
43 function partition(index, arr, subSum, buckets) {
44   if (index === arr.length) {
45     return true;
46   }
47
48   const select = arr[index];
49
50   for (let i = 0; i < buckets.length; i++) {
51     if (i > 0 && buckets[i] === buckets[i - 1]) continue;
52     if (buckets[i] + select <= subSum) {
53       buckets[i] += select;
54       if (partition(index + 1, arr, subSum, buckets)) return true;
55       buckets[i] -= select;
56     }
57   }
58
59   return false;
60 }
```

Java算法源码

```
1 import java.util.LinkedList;
2 import java.util.Scanner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         int m = sc.nextInt();
9
10        LinkedList<Integer> link = new LinkedList<>();
11        for (int i = 0; i < m; i++) {
12            link.add(sc.nextInt());
13        }
14
15        System.out.println(solution(link, m));
16    }
17
18    public static int solution(LinkedList<Integer> link, int m) {
19        link.sort((a, b) -> b - a);
20
21        int sum = 0;
22        for (Integer ele : link) {
23            sum += ele;
24        }
25
26        while (m >= 1) {
27            // 由于canPartition方法中会删除Link元素，因此我们不能直接传递Link过去，需要传递Link备份，否则会影响下一次count判断
28            LinkedList<Integer> link_cp = new LinkedList<>(link);
29            if (canPartitionMSubsets(link_cp, sum, m)) return sum / m;
30            m--;
31        }
32
33        return sum;
34    }
35}
```

```

36 | public static boolean canPartitionMSubsets(LinkedList<Integer> link, int sum, int m) {
37 |
38 |     if (sum % m != 0) return false;
39 |     int subSum = sum / m;
40 |
41 |     if (subSum < link.get(0)) return false;
42 |
43 |     // while (link.get(0) == subSum) { // 此段代码可能会出现越界
44 |     while (!link.isEmpty() && link.get(0) == subSum) {
45 |         link.removeFirst();
46 |         m--;
47 |     }
48 |
49 |     int[] buckets = new int[m];
50 |     return partition(link, 0, buckets, subSum);
51 | }
52 |
53 | public static boolean partition(LinkedList<Integer> link, int index, int[] buckets, int subSum) {
54 |     if (index == link.size()) return true;
55 |
56 |     int select = link.get(index);
57 |
58 |     for (int i = 0; i < buckets.length; i++) {
59 |         if (i > 0 && buckets[i] == buckets[i - 1]) continue;
60 |         if (select + buckets[i] <= subSum) {
61 |             buckets[i] += select;
62 |             if (partition(link, index + 1, buckets, subSum)) return true;
63 |             buckets[i] -= select;
64 |         }
65 |     }
66 |
67 |     return false;
68 | }
69 | }

```

Python算法源码

```
1 # 输入获取
2 m = int(input())
3 link = list(map(int, input().split()))
4
5
6 # 算法入口
7 def getResult(link, m):
8     link.sort(reverse=True)
9
10    sumV = 0
11    for ele in link:
12        sumV += ele
13
14    while m >= 1:
15        if canPartitionMSubsets(link[:], sumV, m):
16            return int(sumV / m)
17        m -= 1
18
19    return sumV
20
21
22 def canPartitionMSubsets(link, sumV, m):
23     if sumV % m != 0:
24         return False
25
26     subSum = sumV / m
27
28     if subSum < link[0]:
29         return False
30
31     while len(link) > 0 and link[0] == subSum:
32         link.pop(0)
33         m -= 1
34
35     buckets = [0] * m
```

```

36 |
37 |     return partition(link, 0, buckets, subSum)
38 |
39 |
40 | def partition(link, index, buckets, subSum):
41 |     if index == len(link):
42 |         return True
43 |
44 |     select = link[index]
45 |
46 |     for i in range(len(buckets)):
47 |         if i > 0 and buckets[i] == buckets[i - 1]:
48 |             continue
49 |
50 |         if select + buckets[i] <= subSum:
51 |             buckets[i] += select
52 |             if partition(link, index + 1, buckets, subSum):
53 |                 return True
54 |             buckets[i] -= select
55 |
56 |     return False
57 |
58 |
59 | # 算法调用
60 | print(getResult(link, m))

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <stdbool.h>
4 |
5 | #define MAX_M 51
6 |
7 | bool partition(const int link[], int link_size, int index, int buckets[], int buckets_size, int subSum) {

```

```

8 |     if (index == link_size) {
9 |         return true;
10 |     }
11 |
12 |     int select = link[index];
13 |
14 |     for (int i = 0; i < buckets_size; i++) {
15 |         if (i > 0 && buckets[i] == buckets[i - 1]) continue;
16 |
17 |         if (select + buckets[i] <= subSum) {
18 |             buckets[i] += select;
19 |             if (partition(link, link_size, index + 1, buckets, buckets_size, subSum)) {
20 |                 return true;
21 |             }
22 |             buckets[i] -= select;
23 |         }
24 |     }
25 |
26 |     return false;
27 | }
28 |
29 | bool canPartitionMSubsets(int link[], int link_size, int sum, int buckets_size) {
30 |     if (sum % buckets_size != 0) return false;
31 |
32 |     int subSum = sum / buckets_size;
33 |     if (subSum < link[0]) return false;
34 |
35 |     int buckets[MAX_M] = {0};
36 |     return partition(link, link_size, 0, buckets, buckets_size, subSum);
37 | }
38 |
39 | int cmp(const void *a, const void *b) {
40 |     return *((int *) b) - *((int *) a);
41 | }
42 |
43 | int solution(int link[], int link_size) {
44 |     int sum = 0;

```



```

45     for (int i = 0; i < link_size; i++) {46         sum += link[i];
47     }
48
49     qsort(link, link_size, sizeof(link[0]), cmp);
50
51     int buckets_size = link_size;
52     while (buckets_size >= 1) {
53         int link_cp[MAX_M];
54         for (int i = 0; i < link_size; i++) link_cp[i] = link[i];
55
56         if (canPartitionMSubsets(link_cp, link_size, sum, buckets_size)) {
57             return sum / buckets_size;
58         }
59
60         buckets_size--;
61     }
62 }
63
64 int main() {
65     int m;
66     scanf("%d", &m);
67
68     int link[MAX_M] = {0};
69     for (int i = 0; i < m; i++) {
70         scanf("%d", &link[i]);
71     }
72
73     printf("%d\n", solution(link, m));
74
75     return 0;
76 }

```

```

1  #include <bits/stdc++.h> 2
3  using namespace std;
4
5  bool partition(deque<int> &link, int index, vector<int> &buckets, int subSum) {
6      if (index == link.size()) return true;
7
8      int select = link[index];
9
10     for (int i = 0; i < buckets.size(); i++) {
11         if (i > 0 && buckets[i] == buckets[i - 1]) continue;
12
13         if (select + buckets[i] <= subSum) {
14             buckets[i] += select;
15             if (partition(link, index + 1, buckets, subSum)) return true;
16             buckets[i] -= select;
17         }
18     }
19
20     return false;
21 }
22
23 bool canPartitionMSubsets(deque<int> &link, int sum, int m) {
24     if (sum % m != 0) return false;
25
26     int subSum = sum / m;
27
28     if (subSum < link[0]) return false;
29
30     while (!link.empty() && link[0] == subSum) {
31         link.pop_front();
32         m--;
33     }
34
35     vector<int> buckets(m, 0);
36     return partition(link, 0, buckets, subSum);
37 }
38

```

```

39 | int solution(deque<int> &link, int m) {
    |                                     40 |     sort(link.begin(), link.end());
41 |     reverse(link.begin(), link.end());
42 |
43 |     int sum = 0;
44 |     for (const auto &item: link) {
45 |         sum += item;
46 |     }
47 |
48 |     while (m >= 1) {
49 |         deque<int> link_cp = link;
50 |         if (canPartitionMSubsets(link_cp, sum, m)) return sum / m;
51 |         m--;
52 |     }
53 |
54 |     return sum;
55 | }
56 |
57 | int main() {
58 |     int m;
59 |     cin >> m;
60 |
61 |     deque<int> link(m);
62 |     for (int i = 0; i < m; i++) {
63 |         cin >> link[i];
64 |     }
65 |
66 |     cout << solution(link, m) << endl;
67 |
68 |     return 0;
69 | }

```